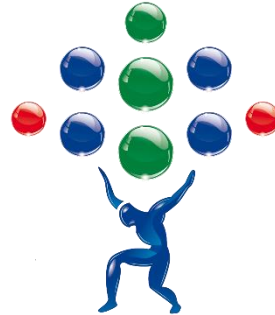


UNIVERSIDAD PRIVADA DOMINGO SAVIO

Ingeniería de Sistemas



UNIVERSIDAD PRIVADA
DOMINGO SAVIO

TRABAJO:

Actividad 9: Retrospectiva Sprint 1 –
"Evolución del Squad con Starfish"

ESTUDIANTE (S):

Josue Adhemar Mamani Huanacu

Josue Matias Arroyo Reynoso

Luis Fernando Arias Segovia

DOCENTE:

Ing. Nelson Huanca Victoria

Profesionales + humanos

1. RESUMEN DEL SPRINT 1 - ALCANCE LOGRADO

El presente Sprint 1 tuvo como objetivo central la construcción del núcleo de seguridad del sistema, abarcando los módulos de registro de usuarios, autenticación y control de acceso basado en roles. Las tres Historias de Usuario planificadas fueron completadas al 100%, cumpliendo en su totalidad con los criterios de aceptación establecidos en la Definition of Done (DoD) del equipo.

HU-01: Registro de Usuarios

Se implementó el endpoint API correspondiente al registro de nuevos usuarios en el sistema. La lógica de negocio fue desarrollada sobre el controlador AuthController, el cual gestiona la validación de los datos de entrada mediante las reglas nativas del framework Laravel 11.

Las validaciones aplicadas garantizan la integridad de los datos antes de su persistencia en la base de datos:

- Campo correo electrónico: validación de unicidad en la tabla de usuarios (unique), formato RFC válido y campo obligatorio (required).
- Campo contraseña: validación de campo obligatorio y confirmación de contraseña mediante la regla confirmed, la cual verifica la coincidencia entre los campos password y password_confirmation.
- Campos generales: todos los campos obligatorios son verificados antes de iniciar el proceso de persistencia.

El cifrado de la contraseña fue implementado mediante la fachada `Hash::make()` de Laravel, la cual aplica el algoritmo de hashing criptográfico Bcrypt de manera irreversible. Esto garantiza que en ningún escenario las contraseñas en texto plano sean almacenadas o recuperadas desde la base de datos.

HU-02: Inicio de Sesión (Login)

Se desarrollaron los flujos de autenticación segura del sistema. El proceso de login valida las credenciales del usuario en dos etapas diferenciadas:

- Primera etapa: verificación de existencia del usuario mediante su correo electrónico registrado.
- Segunda etapa: comprobación del estado de la cuenta antes del procesamiento de credenciales, implementada a través de un Local Scope denominado scopeActivos en el modelo User.

El Local Scope `scopeActivos()` encapsula la condición de filtrado `estado = 1` en un único punto del modelo, evitando la duplicación de esta lógica en múltiples controladores y garantizando que las cuentas inactivas sean bloqueadas de forma sistemática antes de validar las credenciales.

Tras la autentificación exitosa, el sistema genera un token de acceso mediante Laravel Sanctum (`createToken()`). Dicho token es retornado al cliente bajo el esquema Bearer Token, el cual debe ser incluido en la cabecera Authorization de cada solicitud subsiguiente a rutas protegidas.

HU-03: Control de Acceso al DSS

Se implementó la lógica de Control de Acceso Basado en Roles (RBAC) para proteger los módulos analíticos del Sistema de Soporte a la Decisión (DSS). La restricción de acceso fue encapsulada en el método de negocio `hasAccessToDSS()` del modelo User.

El método `hasAccessToDSS()` aplica una condición lógica estricta de conjunción: el usuario debe cumplir simultáneamente con Rol = admin Y Estado = activo. Si cualquiera de las dos condiciones falla, el acceso a las rutas del DSS es denegado con una respuesta HTTP 403 Forbidden.

Esta lógica fue implementada en el modelo de dominio (no en el controlador), siguiendo el principio de separación de responsabilidades y manteniendo la cohesión de las reglas de negocio en la capa de modelo.

2. CONCORDANCIA CON EL DISEÑO DE ARQUITECTURA Y UML

El incremento desarrollado durante el Sprint 1 fue implementado en estricta concordancia con el diagrama de clases preestablecido en la fase de diseño arquitectónico. Los artefactos de código producidos reflejan fielmente las especificaciones UML acordadas por el equipo.

2.1 Correspondencia entre el Diagrama de Clases y el Código Fuente

Los archivos centrales del incremento son los siguientes:

- `app/Models/User.php`: contiene el modelo de dominio con los métodos de negocio y la configuración de seguridad de la entidad usuario.
- `app/Http/Controllers/Api/AuthController.php`: contiene los métodos de registro y autenticación expuestos como endpoints de la API RESTful.

El diagrama de clases definió las siguientes firmas de métodos de negocio en la entidad User, las cuales fueron implementadas sin alteración de sus nombres ni tipos de retorno:

Método (Firma UML)	Tipo de Retorno	Responsabilidad
<code>isAdmin()</code>	bool	Verifica si el rol del usuario es 'admin'

hasAccessToDSS()	bool	Verifica acceso al DSS (rol admin Y estado activo)
getFullName()	string	Retorna la concatenacion del nombre y apellido del usuario

2.2 Proteccion contra Asignacion Masiva (Mass-Assignment)

El campo **estado** fue deliberadamente excluido de la propiedad **\$fillable** del modelo User. Esta decision de diseno constituye una medida de seguridad critica, denominada Mass-Assignment Protection, que impide que un actor malicioso pueda alterar el estado de una cuenta directamente a traves de los datos enviados en una peticion HTTP.

El estado de una cuenta unicamente puede ser modificado mediante operaciones administrativas explicitas y controladas en la capa de servicio, nunca mediante asignacion masiva a traves de **User::create()** o **User::update()** con datos de entrada del usuario.

2.3 Confidencialidad mediante la Propiedad \$hidden

Los campos **password** y **remember_token** fueron incluidos en la propiedad **\$hidden** del modelo User. Esto garantiza que, independientemente del endpoint que retorne datos del usuario, estos campos criticos sean automaticamente omitidos de las respuestas JSON, eliminando el riesgo de exposicion accidental de credenciales.

3. MATRIZ DE CUMPLIMIENTO DEL DEFINITION OF DONE (DoD)

La siguiente matriz documenta el nivel de cumplimiento de cada criterio establecido en el Definition of Done del equipo para el Sprint 1. Todos los criterios fueron verificados y aprobados previo al cierre formal del sprint.

Criterio DoD	Detalle de Verificacion	Estado	Evidencia
Criterios de Aceptacion	Validados al 100% en ambiente de integracion mediante la herramienta Hoppscotch. Codigos HTTP verificados: 200 (OK), 201 (Created), 422 (Unprocessable), 401 (Unauthorized).	CUMPLIDO	Capturas en /docs/test-results
Linter y Formato deCodigo	Codigo revisado y conforme al estandar de codificacion PSR-12 para PHP,	CUMPLIDO	Revision en Pull Request

	garantizando legibilidad y consistencia en todo el incremento.		
Diagrama UML Actualizado	Diagrama de secuencia actualizado para reflejar los flujos de registro, login y control de acceso. Archivo resguardado en la carpeta /docs del repositorio.	CUMPLIDO	/docs/sequence-sprint1.png
Revision de Par (Peer Review)	El codigo fue auditado, comentado y aprobado por un miembro diferente del squad mediante un Pull Request formal en el repositorio GitHub del proyecto.	CUMPLIDO	PR aprobado en GitHub
Documentacion de Endpoints	Archivo README.md actualizado con la especificacion completa de los endpoints: metodos HTTP, rutas, parametros de entrada, formatos de respuesta y codigos de error.	CUMPLIDO	README.md en rama main

4. ALINEACION CON LA NORMA DE CALIDAD ISO 25010

La norma ISO/IEC 25010 define el modelo de calidad del producto de software. A continuacion se argumenta tecnicamente como el incremento del Sprint 1 cumple con los pilares relevantes de dicho estandar.

4.1 Adecuacion Funcional

La sub-caracteristica de Correccion Funcional establece que el software debe proveer resultados correctos con el nivel de precision requerido por las especificaciones del negocio.

El incremento cumple este pilar en los siguientes aspectos:

- HU-01 implementa las reglas de validacion exactas del dominio de repuestos automotrices: correo electronico como identificador unico de usuario y contrasena confirmada antes del registro.
- HU-02 implementa el flujo de autentificacion conforme al proceso de negocio: un usuario inactivo no puede iniciar sesion, replicando la politica administrativa del negocio.

- HU-03 restringe el acceso a los reportes del DSS (tablas de ventas, inventario y facturación) exclusivamente a administradores activos, respetando la jerarquía de roles del negocio.

4.2 Seguridad - Confidencialidad e Integridad

La sub-característica de Confidencialidad exige que la información sea accesible únicamente a entidades con autorización. La sub-característica de Integridad exige que el sistema prevenga el acceso y modificación no autorizados.

El incremento cumple este pilar mediante cuatro mecanismos técnicos implementados:

- Hashing criptográfico irreversible: la implementación de `Hash::make()` con `Bcrypt` garantiza que las contraseñas no puedan ser recuperadas ni revertidas en ningún escenario, incluso ante una brecha de datos en la base de datos.
- Ocultamiento de campos sensibles: la propiedad `$hidden` del modelo `User` asegura que los campos `password` y `remember_token` sean excluidos de todas las respuestas JSON de la API, previniendo su exposición accidental.
- Autenticación con Bearer Tokens: la firma de peticiones mediante tokens `Sanctum` garantiza que solo clientes autenticados puedan consumir los endpoints protegidos, previniendo accesos no autorizados.
- Protección Mass-Assignment: la exclusión del campo estado de `$fillable` previene que un actor malicioso pueda escalar privilegios o activar cuentas bloqueadas mediante inyección de datos en las peticiones HTTP.

4.3 Mantenibilidad - Modularidad

La sub-característica de Modularidad establece que el sistema debe estar compuesto de componentes discretos, de forma que un cambio en un componente tenga un impacto mínimo en los demás.

El incremento cumple este pilar mediante el uso de Local Scopes de Laravel:

- El Local Scope `scopeActivos()` centraliza la condición de filtrado por estado activo en un único punto del modelo `User`. Si en el futuro el criterio de activación de una cuenta cambia (por ejemplo, incorporar un estado de cuenta 'en revisión'), el cambio se realiza en un solo lugar sin necesidad de modificar ningún controlador.
- Los métodos de negocio `isAdmin()`, `hasAccessToDSS()` y `getFullName()` encapsulan reglas que podrían volverse más complejas en sprints futuros. Al estar en el modelo, cualquier controlador o middleware puede invocarlos sin duplicar lógica.
- El `AuthController` se mantiene delgado (thin controller), delegando la lógica de dominio al modelo, lo cual facilita las pruebas unitarias independientes de cada capa.

5. RETROSPECTIVA DEL SPRINT 1 - DINAMICA STARFISH

Al cierre del Sprint 1, el equipo condujo la retrospectiva formal denominada 'Evolucion del Squad con Starfish', conforme a la Actividad 9 del curso. La sesion se desarrollo en cuatro etapas: apertura con seguridad psicologica, dinamica de la estrella de cinco puntas, priorizacion mediante votacion, y transferencia de las mejoras al Sprint Backlog del Sprint 2 (Scrumming the Scrum).

5.1 Apertura y Seguridad Psicologica

El Scrum Master dio inicio a la sesion leyendo la Directiva Primaria de la retrospectiva, estableciendo un marco de confianza para el analisis critico:

“Independientemente de lo que descubramos, entendemos y creemos sinceramente que todos hicieron el mejor trabajo posible”

Como dinamica rompehielos, el equipo aplico la tecnica del Termometro, en la cual cada integrante expreso de forma rapida su estado animico previo al analisis. El resultado general reflejo un animo positivo y de satisfaccion, atribuible al cierre exitoso del nucleo de seguridad del sistema.

5.2 Tablero Starfish - Aportes del Equipo por Categoria

Cada miembro del Squad apporto ideas en las cinco categorias de la dinamica Starfish. El tablero fue elaborado de forma digital y resguardado en la Wiki del repositorio GitHub del proyecto. A continuacion se documentan los aportes consolidados:

Categoria	Aportes del Squad
Keep Doing (Mantener)	Mantener la cultura de Peer Reviews mediante Pull Requests en GitHub, que permitio detectar defectos antes de fusionar codigo. Mantener las pruebas rigurosas de endpoints en Hoppscotch, verificando sistematicamente los codigos HTTP 200, 201, 422 y 401 en ambiente de integracion. Mantener la actualizacion paralela del README.md con la especificacion de endpoints durante el desarrollo, evitando deuda de documentacion.
Less Of (Menos de)	Menos reuniones de coordinacion improvisadas durante las horas de desarrollo, las cuales fragmentaron el tiempo de concentracion (deep work) de los integrantes en mas de una ocasion durante el sprint. Menos retrabajo causado por iniciar la codificacion de un endpoint antes de que el diagrama de secuencia correspondiente estuviera finalizado.
More Of (Mas de)	Mas rigurosidad en documentar los modelos UML a tiempo, ya que el diagrama de secuencia de HU-02 y HU-03 fue actualizado con posterioridad al codigo en lugar de de forma concurrente. Mas comunicacion proactiva entre los responsables del modelo User.php y del

	AuthController.php para anticipar cambios en las firmas de los metodos de negocio.
Stop Doing (Dejar de hacer)	Dejar de programar sin consultar previamente los criterios de aceptacion definidos en la Historia de Usuario, practica que en HU-02 genero un retrabajo menor al implementar primero la logica de bloqueo de cuentas y validarla recien despues contra el criterio formal. Dejar de fusionar cambios a la rama principal sin la aprobacion explicita de al menos un Peer Review.
Start Doing (Empezar a hacer)	Empezar a aplicar la tecnica de Pomodoro grupal durante las sesiones de laboratorio, con bloques de enfoque y pausas sincronizadas entre los tres integrantes. Empezar a refinar el Backlog del Sprint 2 con mayor especificidad en los criterios de aceptacion antes del Sprint Planning, evitando ambigüedades como las observadas en HU-02 y HU-03.

5.3 Priorizacion y Votacion

Conforme al protocolo establecido, cada uno de los tres integrantes del Squad distribuyo 3 puntos de votacion entre los aportes considerados mas criticos para el exito del Sprint 2. El resultado de la votacion (9 puntos totales distribuidos) determino las dos mejoras prioritarias a transformarse en acciones SMART:

- 1ra prioridad (4 puntos): Mayor rigurosidad en la documentacion de modelos UML a tiempo (categoria More Of).
- 2da prioridad (3 puntos): Definicion de criterios de aceptacion con mayor especificidad antes del Sprint Planning (categoria Start Doing).
- Otros aportes con menor votacion (2 puntos en conjunto): adopcion de Pomodoro grupal, quedando registrada como practica a evaluar de forma informal sin convertirse en tarea formal del Backlog.

5.4 Plan de Accion - Mejoras SMART para el Sprint 2

Las dos mejoras mas votadas fueron convertidas en tareas SMART (Especificas, Medibles, Alcanzables, Relevantes y con Tiempo definido), conforme al Informe de Decisiones Tomadas exigido en la actividad. Cada mejora cuenta con un guardian (responsable) asignado del Squad:

Que vamos a mejorar	Como sabremos que funciona	Definicion SMART	Guardian
Actualizar el diagrama UML de secuencia de forma concurrente al desarrollo del codigo, no posterior a este.	Al cierre del Sprint 2, el 100% de las Historias de Usuario completadas tendran su diagrama de secuencia mergeado	Especifica: diagramas de secuencia por HU. Medible: porcentaje de HU con diagrama a tiempo. Alcanzable: tarea de 30 min por	Josue Matias Arroyo Reynoso

	en la carpeta /docs en la misma fecha o antes que el Pull Request del código correspondiente.	HU. Relevante: reduce retrabajo. Tiempo: durante todo el Sprint 2.	
Refinar los criterios de aceptación de cada Historia de Usuario con mayor especificidad antes del Sprint Planning del Sprint 2.	Al iniciar el Sprint Planning del Sprint 2, el 100% de las HU del Backlog contarán con criterios de aceptación redactados en formato Given-When-Then, revisados y aprobados por el Scrum Master.	Específica: formato Given-When-Then. Medible: porcentaje de HU con criterios formalizados. Alcanzable: sesión de refinamiento de 1 hora. Relevante: reduce ambigüedad técnica. Tiempo: antes del Sprint Planning.	Luis Fernando Arias Segovia

5.5 Scrumming the Scrum - Transferencia al Sprint Backlog

En cumplimiento del Paso 4 de la actividad, ambas mejoras fueron formalizadas como tarjetas de tarea dentro del Sprint Backlog del Sprint 2, asegurando que no queden como acuerdos informales sino como compromisos verificables dentro del ciclo de trabajo:

- Tarjeta 1: 'Actualizar diagramas de secuencia de forma concurrente al código' - asignada a Josue Matias Arroyo Reynoso, columna To Do del Sprint 2.
- Tarjeta 2: 'Formalizar criterios de aceptación en formato Given-When-Then antes del Planning' - asignada a Luis Fernando Arias Segovia, columna To Do del Sprint 2.

Enlace a las tareas de mejora en el tablero de GitHub Projects del Sprint 2:
<https://github.com/users/Haxlit/projects/5>

5.6 Analisis Sistemico - Vinculo con el Manifiesto Agil e ISO 25010

Más allá de la identificación superficial de aciertos y fallos, el Squad reconoce que los patrones observados en el Sprint 1 se conectan directamente con principios de ingeniería de software de orden superior:

- La práctica de Peer Reviews (Keep Doing) materializa el principio del Manifiesto Agil de atención continua a la excelencia técnica, y refuerza directamente la característica de Mantenibilidad de la norma ISO 25010, al prevenir la introducción de código de baja calidad en el repositorio.
- El retraso en la actualización de los diagramas UML (More Of) representa un riesgo para la sub-característica de Trazabilidad y Comprensibilidad de la Mantenibilidad según ISO 25010, dado que un modelo desactualizado dificulta el análisis de impacto ante futuros cambios.

- La interrupcion del flujo de trabajo por reuniones improvisadas (Less Of) atenta contra el principio agil de procesos sostenibles que el equipo pueda mantener indefinidamente, afectando la productividad sin generar valor proporcional.
- Programar sin consultar los criterios de aceptacion (Stop Doing) compromete directamente la sub-caracteristica de Adecuacion Funcional de ISO 25010, ya que el software puede implementarse de forma tecnicamente correcta pero funcionalmente desalineada con la necesidad real del negocio.

Conclusion Final del Sprint 1

El Sprint 1 ha concluido de forma exitosa. El nucleo de seguridad del Sistema DSS de Ventas de Repuestos Automotrices y Facturacion Electronica se encuentra cerrado, verificado y documentado al 100%. Los mecanismos de registro seguro, autenticacion con tokens y control de acceso basado en roles constituyen los cimientos sobre los cuales se construiran los modulos transaccionales del sistema.

Con este nucleo estable y auditado, el proyecto se encuentra en condiciones optimas para iniciar el Sprint 2, el cual abordara el desarrollo de los modulos transaccionales de gestion de inventarios de repuestos automotrices y el modulo de ventas, ambas funcionalidades core del sistema de soporte a la decision.

Conforme al principio de mejora continua del 1% planteado en la dinamica Starfish, el Squad adopta de forma deliberada mejoras incrementales y acotadas en lugar de cambios disruptivos, privilegiando la sostenibilidad del ritmo de trabajo del equipo a lo largo de los proximos sprints.

6. FIRMAS Y APROBACION

Los integrantes del squad certifican que el contenido del presente informe tecnico refleja fielmente el trabajo realizado durante el Sprint 1 y que todos los artefactos descritos se encuentran disponibles en el repositorio del proyecto.

Estudiante	Estudiante	Estudiante
Josue Adhemar Mamani Huanacu	Josue Matias Arroyo Reynoso	Luis Fernando Arias Segovia

Docente Supervisor

Ing. Nelson Huanca Victoria